

DOCUMENTACIÓN DEL PROYECTO

Documentación integral de GymTrack

Arquitectura, funcionamiento, seguridad y experiencia de uso.

EDICIÓN

Agosto de 2026

UBICACIÓN

Montevideo, Uruguay

ESTADO

Versión funcional 1.0

EQUIPO DEL PROYECTO

Leandro González

Liderazgo

Máximo Díaz

Backend

Emilio Escobar

Frontend

Santiago Cáceres

Base de datos

Magdalena Belmonti

Documentación

Hiliana Pereira

Documentación

Un producto explicado desde la experiencia hasta la base de datos

Esta edición une el planteo académico de Magdalena Belmonti e Hiliana Pereira con el funcionamiento comprobable del repositorio. No es un inventario de pantallas: explica decisiones, recorridos, límites y aprendizajes.

Flujos estables	Integraciones condicionadas	Beta o desactivado
Identidad, catálogo, administración, agenda, pagos manuales, finanzas, reportes y experiencia del socio.	Mercado Pago, correo y Turnstile requieren credenciales, red y un entorno equivalente al real.	WhatsApp, Calendar OAuth, analítica avanzada y recomendaciones no se presentan como operaciones terminadas.

Cuatro recorridos para entender GymTrack

01 · Producto Problema, objetivos, público y alcance.	02 · Ingeniería Frontend, PHP, API, MySQL y seguridad.
03 · Operación Roles, reservas, pagos, finanzas y comunicación.	04 · Aprendizaje Problemas reales, riesgos preventivos y trabajo del equipo.
Principio rector Una acción se considera terminada sólo cuando el servidor la autoriza y el resultado puede comprobarse.	

Responsabilidades: Leandro González, liderazgo; Máximo Díaz, backend; Emilio Escobar, frontend; Santiago Cáceres, base de datos; Magdalena Belmonti e Hiliana Pereira, documentación.

Índice general

Treinta y siete capítulos recorren el producto, su arquitectura, la operación cotidiana y las decisiones que permitieron estabilizarlo.

01	Introducción	20	Promociones, notificaciones y calendario
02	Descripción de GymTrack	21	Mapa y geolocalización
03	Problema que resuelve	22	Sistema de demostración
04	Objetivos	23	Diseño e identidad visual
05	Alcance y estado real	24	Paleta de colores
06	Público objetivo	25	Tipografía
07	Equipo de trabajo	26	Sistema de logos
08	Tecnologías utilizadas	27	Responsive y accesibilidad
09	Arquitectura del sistema	28	Pruebas y control de calidad
10	Frontend	29	Problemas encontrados y soluciones
11	Backend y API	30	Trabajo del equipo y trazabilidad
12	Base de datos	31	Herramientas de trabajo utilizadas
13	Seguridad	32	Evolución del proyecto
14	Roles y permisos	33	Limitaciones actuales
15	Funcionalidades	34	Mejoras futuras
16	Flujos de uso	35	Instalación, demostración y producción
17	Administración	36	Conclusiones
18	Experiencia del socio	37	Fuentes verificadas
19	Pagos, finanzas y reportes		

01 · Introducción

GymTrack es una plataforma web orientada a dos necesidades relacionadas: permitir que una persona descubra gimnasios y ofrecer a cada establecimiento una herramienta centralizada para administrar su operación. El producto combina una experiencia pública, un área personal para socios y una sección administrativa multi-gimnasio.

El proyecto nació de una situación cotidiana: en muchos gimnasios pequeños y medianos, la información importante termina repartida entre mensajes, papel y hojas de cálculo. Una persona pregunta por WhatsApp si su cuota está al día; otra anota una reserva; alguien más corrige a mano un cupo. Cada herramienta resuelve algo en el momento, pero el conjunto se vuelve difícil de controlar cuando aumentan los socios, las clases y los pagos. GymTrack busca que esas tareas dejen de depender de la memoria de una persona y pasen a tener estados, permisos e historial.

La documentación inicial elaborada por Magdalena Belmonti e Hiliana Pereira aportó la formulación académica del problema, los objetivos, los primeros veinte requisitos funcionales, la planificación general, la identidad visual y el modelo de datos previsto. A partir de esa base, este documento incorpora el estado comprobable del sistema actual. La arquitectura original de Vue, PHP, MySQL y Docker se conservó, pero el alcance creció con autenticación verificable, permisos por gimnasio, agenda fechada, lista de espera, pagos, finanzas, reportes, promociones, notificaciones, geolocalización y funciones personales del socio.

La versión actual está preparada para una presentación funcional. Los flujos principales están conectados a MySQL y no dependen de datos estáticos de Vue. Algunas integraciones externas permanecen condicionadas por credenciales o desactivadas mediante banderas; se identifican con precisión en este documento.

Este texto está pensado para dos lecturas. Una persona no técnica puede seguir el problema, los objetivos, los actores y los recorridos de uso. Quien necesite mantener el proyecto encontrará además la estructura interna, las fronteras de seguridad, las relaciones de datos, los estados de negocio, los problemas resueltos y los controles operativos pendientes.

02 · Descripción de GymTrack

GymTrack organiza la información alrededor de un gimnasio activo. Un visitante puede explorar el catálogo y el mapa. Un socio puede consultar su membresía, reservar clases, revisar pagos, editar su perfil y presentar un carné digital. Empleados y dueños acceden a una navegación administrativa cuyas opciones dependen de sus permisos efectivos. El administrador general puede operar en modo de soporte dentro del conjunto de datos correspondiente.

La aplicación mantiene tres superficies principales:

- **Experiencia pública:** inicio, catálogo geográfico, ficha de gimnasio, planes y solicitud de alta para dueños.
- **Área del socio:** resumen, agenda, pagos, progreso, perfil, preferencias, favoritos, medidas y carné digital.
- **Administración:** resumen, gestión operativa, personas, entrenadores, clases, reservas, membresías, pagos, promociones, finanzas, reportes y configuración.

La fuente de verdad de la operación es el backend PHP con MySQL. El frontend no aprueba pagos, no decide permisos y no selecciona libremente el alcance de un gimnasio: esas decisiones se vuelven a validar en el servidor.

03 · Problema que resuelve

La transformación digital cambió la forma en que las organizaciones administran información y toman decisiones. Sin embargo, una parte importante de los gimnasios de pequeña y mediana escala todavía depende de registros físicos, hojas de cálculo y aplicaciones de mensajería para coordinar actividades críticas. Estas herramientas son útiles para resolver necesidades inmediatas, pero no fueron pensadas para mantener una operación integral, concurrente y trazable.

Cuando la información está dispersa, el problema no es solamente “tener muchos archivos”. También se vuelve difícil saber cuál registro es el correcto, quién realizó un cambio y qué debería ocurrir después. Una membresía puede vencer sin aviso, una reserva puede ocupar el último cupo dos veces, un pago puede quedar sin vincular o una promoción puede enviarse a alguien que pidió no recibir publicidad.

Las dificultades principales, ya identificadas en el documento inicial y confirmadas como fundamento del producto, son:

- membresías sin un estado centralizado;
- pagos sin historial verificable;
- sobrecupos y reservas duplicadas;
- comunicación sin preferencias ni trazabilidad;
- falta de separación entre el trabajo del dueño, el empleado y el socio;
- poca visibilidad sobre deuda, ingresos y asistencia;
- crecimiento operativo difícil de sostener con herramientas informales.

GymTrack responde mediante datos relacionales, permisos explícitos, transacciones, auditoría y estados visibles. El objetivo no es sustituir todas las herramientas del gimnasio, sino concentrar los procesos críticos en una aplicación web verificable.

En términos humanos, el beneficio buscado es sencillo: que un socio pueda resolver sus gestiones sin depender de una respuesta manual y que el personal pueda trabajar con la tranquilidad de que una acción queda registrada, autorizada y vinculada al gimnasio correcto.

04 · Objetivos

4.1 Objetivo general

Centralizar la gestión cotidiana de gimnasios y la experiencia digital de sus socios mediante una aplicación web segura, responsiva y preparada para operar con más de un gimnasio sin mezclar información.

4.2 Objetivos específicos alcanzados

1. Registrar usuarios, verificar su correo, autenticar mediante sesión y permitir recuperar o cambiar la contraseña.
2. Aplicar roles globales y permisos efectivos por gimnasio desde PHP.
3. Publicar gimnasios reales desde MySQL con sedes, horarios, categorías, servicios y ubicación.
4. Gestionar socios, empleados, entrenadores, planes y membresías.
5. Crear clases recurrentes y sesiones fechadas con cupos, reservas, lista de espera y asistencia.
6. Registrar pagos manuales e integrar un proveedor desacoplado para checkout y webhooks.
7. Obtener finanzas y reportes desde datos persistidos.

8. Gestionar promociones y notificaciones con consentimiento, preferencias e idempotencia.
9. Dar al socio un panel personal con progreso, historial, favoritos, medidas opcionales y carné digital.
10. Mantener un dataset de presentación reproducible, identificable y eliminable.
11. Preparar imágenes y composiciones separadas para desarrollo y producción.

4.3 Criterios de éxito del proyecto

Se considera que un flujo principal está realmente resuelto cuando cumple las siguientes condiciones:

1. la persona puede encontrar la acción desde una navegación coherente con su rol;
2. la interfaz explica qué está ocurriendo mediante carga, error, vacío, éxito o bloqueo;
3. PHP vuelve a validar identidad, permiso, gimnasio y datos recibidos;
4. MySQL conserva el resultado o el historial necesario;
5. una repetición accidental no produce reservas, pagos o mensajes duplicados;
6. el resultado puede verificarse mediante una consulta posterior o un registro de auditoría;
7. si depende de un servicio externo, la aplicación no presenta una intención como una confirmación.

4.4 Trazabilidad de los requisitos funcionales iniciales

La primera documentación numeró veinte requisitos. La tabla siguiente los conserva como vínculo entre el planteo original y la implementación actual. “Ampliado” significa que el requisito se resolvió y luego evolucionó hacia un contrato más completo.

ID	Requisito original resumido	Estado actual	Evidencia funcional
RF-01	Registrar socios con datos personales	Ampliado	Registro público, aceptación legal, Turnstile configurable y perfil del socio.
RF-02	Correo único	Implementado	Restricción única, validación PHP y respuesta de conflicto.
RF-03	Iniciar sesión	Ampliado	Sesión de servidor, cookie HttpOnly , contexto, CSRF y registro de sesiones.
RF-04	Cerrar sesión	Ampliado	Cierre actual/global y revocación.

ID	Requisito original resumido	Estado actual	Evidencia funcional
RF-05	Actualizar nombre y teléfono	Ampliado	Perfil con datos personales, fotografía y validación backend.
RF-06	Consultar estado de membresía	Implementado	Resumen, historial y estados visibles para el socio.
RF-07	Activar, renovar o suspender membresía	Ampliado	Transiciones autorizadas, plan versionado e historial.
RF-08	Registrar inicio y vencimiento	Implementado	Fechas persistidas y consultables por gimnasio.
RF-09	Avisar vencimiento próximo	Implementado	Notificación interna y canal de correo condicionado por transporte.
RF-10	Bloquear reserva sin membresía activa	Implementado	Regla validada en PHP antes de confirmar la reserva.
RF-11	Listar clases, instructor y cupos	Ampliado	Clases recurrentes y sesiones fechadas con disponibilidad real.
RF-12	Reservar cuando existe disponibilidad	Ampliado	Reserva idempotente, cupo transaccional y lista de espera.
RF-13	Descontar cupo de forma atómica	Implementado	Transacción, bloqueo de sesión y restricción única.
RF-14	Cancelar con anticipación	Ampliado	Regla temporal, cancelación y evento de historial.
RF-15	Restaurar cupo al cancelar	Ampliado	Liberación y promoción de la siguiente persona en espera.
RF-16	Administrar clases	Ampliado	Definiciones recurrentes, sesiones, cancelación y roster.
RF-17	Listar, buscar y filtrar usuarios	Ampliado	Socios, empleados y entrenadores limitados al gimnasio activo.
RF-18	Consultar historial de reservas	Implementado	Listados administrativos y eventos asociados.

ID	Requisito original resumido	Estado actual	Evidencia funcional
RF-19	Registrar pagos manuales	Ampliado	Permiso específico, auditoría, historial y proveedor desacoplado.
RF-20	Mostrar estadísticas de ocupación	Ampliado	Resumen administrativo, asistencia, finanzas y reportes reales.

05 · Alcance y estado real

5.1 Clasificación de estado

Estado	Significado en esta documentación
Estable	El flujo tiene interfaz, endpoint y persistencia o resultado verificable.
Condicionado	Está implementado, pero necesita red, credenciales o infraestructura externa.
Beta/desactivado	Existe una interfaz o contrato parcial, pero la función no se presenta como operativa.
Compatibilidad	Se conserva para consumidores heredados; no define la experiencia principal actual.

5.2 Matriz funcional actual

Área	Estado	Evidencia principal	Observación
Catálogo, mapa y filtros	Estable	<code>GymExplorer.vue</code> , API pública y MySQL	Los tiles dependen de OpenStreetMap.
Registro, login y sesiones	Estable	<code>AuthController</code> , <code>SessionManager</code> , store <code>auth</code>	Cookie <code>HttpOnly</code> y CSRF.
Verificación y recuperación	Estable	tokens, correo y rutas dedicadas	El envío real exige transporte de correo.
Roles y permisos	Estable	<code>AuthorizationService</code> ,	PHP es la autoridad final.

Área	Estado	Evidencia principal	Observación
		middleware y tablas RBAC	
Administración multi-gimnasio	Estable	rutas <code>/administracion</code> , contexto en sesión	El menú refleja permisos efectivos.
Personas, entrenadores y membresías	Estable	controladores y repositorios administrativos	Incluye invitaciones y archivos.
Clases, reservas y asistencia	Estable	<code>ScheduleController</code> y <code>ScheduleRepository</code>	Cupos, espera e idempotencia.
Pagos manuales	Estable	<code>PaymentController</code> y auditoría	Limitado por permiso.
Mercado Pago	Condicionado	interfaz de proveedor y adaptador	Requiere credenciales y webhook válido.
Finanzas	Estable	consultas de <code>PaymentRepository</code>	Datos reales del gimnasio activo.
Excel y PDF	Estable	<code>ReportController</code> y escritores propios	PDF es un resumen; Excel conserva detalle.
Promociones	Estable	<code>PromotionController/Repository</code>	Programar, pausar, finalizar y medir.
Notificaciones internas	Estable	inbox, dispatcher y canal interno	Respeta preferencias y consentimiento.
Correo de notificaciones	Condicionado	canal de correo y worker	Requiere transporte SMTP/mail.
Google Calendar manual e ICS	Estable	<code>CalendarController</code> e <code>IcsCalendarService</code>	No existe sincronización OAuth.
Perfil y progreso del socio	Estable	<code>MemberController/Repository</code>	Incluye datos y fotografía.
Carné digital	Estable	token firmado y verificador administrativo	Lectura por pegado o lector USB.

Área	Estado	Evidencia principal	Observación
WhatsApp	Beta, desactivado	interfaz y adaptador de fallo seguro	No hay proveedor de envío.
Sincronización Google OAuth	Beta, desactivada	bandera y mensaje explícito	Enlace e ICS siguen disponibles.
Analítica avanzada	Desactivada	feature flag	Las métricas convencionales sí funcionan.
Recomendaciones personalizadas	Desactivada	feature flag	Favoritos y preferencias sí funcionan.
Textos legales	Provisional	LegalView.vue	Requieren revisión jurídica antes de lanzamiento.

06 · Público objetivo

- **Visitantes:** personas que comparan gimnasios por ubicación, servicios o categoría.
- **Socios:** usuarios asociados a uno o más gimnasios que necesitan reservas, pagos y seguimiento personal.
- **Empleados:** personal operativo con permisos configurables para socios, agenda, pagos u otras tareas.
- **Dueños:** responsables de uno o más gimnasios, su configuración y sus resultados.
- **Administradores generales:** personal de plataforma con acceso de soporte auditado.
- **Gimnasios pequeños y medianos:** establecimientos que necesitan sustituir procesos dispersos por una operación centralizada.

07 · Equipo de trabajo

GymTrack fue desarrollado por un equipo con responsabilidades diferenciadas. La distribución nominal fue confirmada por el equipo para esta versión de la documentación.

Integrante	Responsabilidad principal	Alcance dentro del proyecto
Leandro González	Líder del proyecto	Coordinación general, prioridades, integración entre áreas y preparación de la entrega.
Máximo Díaz	Responsable de backend	API PHP, reglas de negocio, autenticación, autorización, servicios e integraciones del servidor.
Emilio Escobar	Encargado de frontend	Experiencia Vue, componentes, navegación, estados visuales, responsive e integración con la API.
Santiago Cáceres	Responsable de base de datos	Modelo relacional, migraciones, integridad, índices, consultas y consistencia de MySQL.
Magdalena Belmonti	Documentadora	Organización del contenido, planteo académico, requisitos, explicación funcional y revisión de la entrega.
Hiliana Pereira	Documentadora	Relevamiento, estructura documental, descripción del sistema y coherencia de la presentación escrita.

7.1 Cómo se relacionan las responsabilidades

Las áreas no funcionan de forma aislada. Una reserva, por ejemplo, empieza en una interacción diseñada en frontend, continúa en un endpoint y una regla de backend, y termina en una transacción y varias relaciones de base de datos. La documentación toma ese recorrido completo y lo explica para que el resultado no quede reducido a una lista de pantallas.

El liderazgo coordina esas dependencias y ayuda a decidir el orden de trabajo. Backend y base de datos acuerdan qué estados pueden existir y cómo se protege la integridad. Frontend utiliza esos contratos para mostrar acciones posibles y respuestas honestas. Documentación transforma las decisiones y el funcionamiento en material que pueda entender una persona ajena al código.

7.2 Alcance de las atribuciones

Los cargos anteriores describen la responsabilidad principal de cada integrante. No significan que cada archivo haya sido escrito exclusivamente por una sola persona ni permiten reconstruir autoría línea por línea. La implementación es un resultado colectivo; por eso, cuando este documento describe un problema o una solución, lo atribuye al área involucrada y no inventa testimonios personales.

08 · Tecnologías utilizadas

Capa	Tecnología real	Uso
Interfaz	Vue 3, Composition API	Vistas y componentes reactivos.
Estado	Pinia	Sesión, administración, agenda, pagos, promociones, avisos y socio.
Navegación	Vue Router	Rutas, carga diferida y guards de experiencia.
Cliente HTTP	Axios	Comunicación JSON, uploads y descargas.
Build	Vite 8	Desarrollo, pruebas y bundle productivo.
Diseño	CSS, Manrope Variable, Tabler Icons	Sistema visual, responsive y estados.
Mapas	Leaflet 1.9 + OpenStreetMap	Tiles, marcadores, clusters y navegación geográfica.
Backend	PHP 8.2 + Apache	API REST, reglas de negocio y servicios.
Persistencia	PDO + MySQL 8	Consultas preparadas y transacciones.
Infraestructura	Docker y Docker Compose	Desarrollo reproducible y composición productiva.
Servidor frontend	Nginx	SPA y proxy de /api .

Capa	Tecnología real	Uso
productivo		
Pruebas	Vitest, Vue Test Utils, Playwright, Unitarias, UI, contratos e Axe, scripts PHP/Shell	integración.
Integraciones	Mercado Pago, correo, Turnstile, Google Calendar	Flujos condicionados o manuales según el caso.

El backend no usa un framework PHP ni dependencias de Composer. Sus controladores, middleware, repositorios y servicios son código del proyecto apoyado en extensiones estándar como PDO MySQL, cURL, Fileinfo, mbstring y OPcache en producción.

8.1 Por qué se utiliza cada tecnología

Vue 3 se utiliza para construir una interfaz reactiva sin recargar toda la página en cada acción. La Composition API permite agrupar estado y comportamiento por responsabilidad; por ejemplo, un formulario puede reunir carga, validación, envío y manejo de errores sin mezclar esa lógica con otros módulos.

Pinia conserva el estado compartido de la sesión y de cada dominio. Su función no es reemplazar MySQL: los stores mantienen una copia temporal para la experiencia del navegador, mientras que la API sigue siendo la autoridad. Al cambiar de gimnasio o cerrar sesión, los stores sensibles se limpian para evitar que la pantalla muestre información del contexto anterior.

Vue Router representa los recorridos de la aplicación y carga bajo demanda muchas vistas. Sus guards evitan transiciones confusas —por ejemplo, entrar a administración sin iniciar sesión—, pero la seguridad real sigue en PHP porque cualquier persona podría llamar directamente a un endpoint.

Axios centraliza solicitudes HTTP, cookies, token CSRF, identificadores de solicitud y tratamiento de errores. Esto evita que cada componente invente una forma distinta de comunicarse con el backend.

PHP 8.2 recibe las solicitudes, valida la identidad, aplica reglas y coordina la persistencia. Se mantuvo una implementación liviana y explícita, sin framework, para conservar el stack original y hacer visible el recorrido entre router, controlador, servicio y repositorio.

MySQL 8 mantiene la fuente de verdad relacional. Sus claves, índices, restricciones y transacciones protegen invariantes que no deberían depender sólo del código de interfaz, como evitar una reserva duplicada o conservar una referencia de pago única.

Docker reduce diferencias entre equipos de desarrollo y separa frontend, backend y base de datos. Las composiciones de desarrollo y producción no son idénticas: desarrollo prioriza iteración; producción genera artefactos, restringe servicios y espera TLS y secretos gestionados externamente.

Leaflet y OpenStreetMap permiten explorar gimnasios sin introducir un proveedor cartográfico propietario. Leaflet administra el mapa, capas, marcadores y eventos; OpenStreetMap entrega los mosaicos visuales. La ubicación del usuario se procesa localmente en el navegador y no se envía automáticamente a la API.

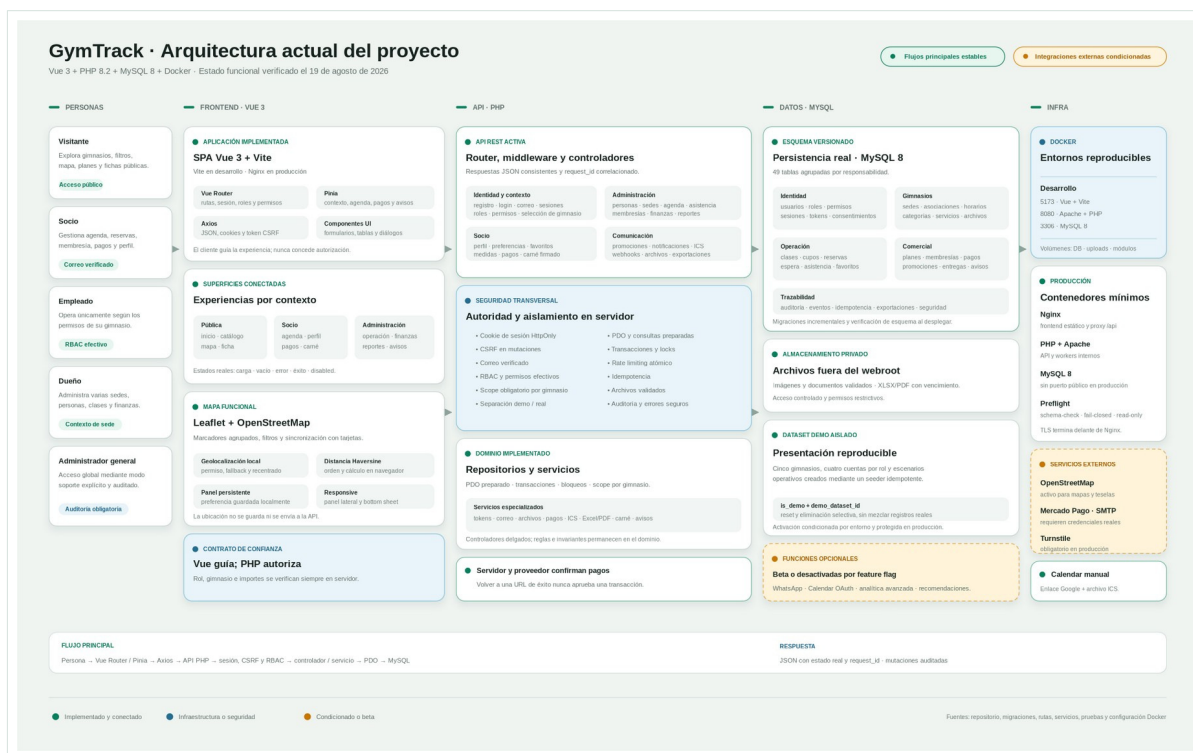
| 8.2 Entorno de trabajo reproducible

La composición de desarrollo levanta tres servicios principales: Vite/Vue, Apache/PHP y MySQL. El frontend llama a `/api`; durante desarrollo el proxy dirige esas solicitudes al backend. En producción, Nginx sirve el bundle estático y reenvía la API a PHP. Un contenedor de comprobación valida migraciones antes de considerar el entorno listo.

Esta separación permite ubicar mejor los fallos: si una vista carga pero una lista no, se revisan red y API; si la API responde pero falla al persistir, se revisan consultas y esquema; si todo funciona en local pero no en despliegue, se comparan variables, volúmenes, proxy y cabeceras.

09 · Arquitectura del sistema

GymTrack usa una arquitectura web por capas. La separación actual ya no es solamente conceptual: cada capa tiene responsabilidades y contratos comprobables.



9.1 Flujo principal de una solicitud

1. La persona interactúa con una vista Vue en el navegador.
2. Vue Router comprueba la condición de navegación y Pinia aporta sesión y contexto visual.
3. Axios envía la solicitud a `/api`, incluye cookies y agrega el token CSRF en mutaciones.
4. Apache entrega la solicitud al punto de entrada PHP y al router de la API.
5. El middleware valida sesión, correo, rol, permiso y contexto de gimnasio.
6. El controlador valida el cuerpo y delega en repositorios o servicios.
7. PDO ejecuta consultas preparadas, transacciones o bloqueos sobre MySQL.
8. La respuesta incluye un estado HTTP, JSON y un `request_id`; las descargas y archivos usan respuestas binarias controladas.

9.2 Componentes arquitectónicos

- **Cliente:** navegador, cookie de sesión, preferencias locales del explorador y geolocalización opcional.
- **Frontend:** Vue, Router, Pinia, Axios, componentes, vistas, Leaflet y QR.
- **API:** router PHP, middleware, controladores y respuesta uniforme.
- **Dominio:** repositorios, proveedores, servicios de correo, tokens, archivos, reportes, calendario, notificaciones y carné.
- **Datos:** MySQL, almacenamiento privado, exportaciones y sesiones persistentes.
- **Procesos:** worker de notificaciones y mantenimiento de tokens/límites.
- **Infraestructura:** Docker Compose, Nginx, Apache/PHP, MySQL, `schema-check`, volúmenes y preflight.

9.3 Integraciones

- OpenStreetMap suministra tiles al navegador.
- Mercado Pago recibe checkout y emite webhooks firmados.
- Correo usa un transporte configurable; en desarrollo puede escribir en log.
- Turnstile protege registro cuando está habilitado.
- Google Calendar se usa mediante URL precompletada e ICS.
- WhatsApp sólo dispone de una interfaz de canal y un adaptador que falla de forma segura.

9.4 Fronteras de confianza

La arquitectura se entiende mejor si se distingue qué partes pueden orientar una acción y cuáles tienen autoridad para confirmarla:

Zona	Puede hacer	No debe decidir por sí sola
Navegador	Mostrar la interfaz, conservar preferencias locales y solicitar acciones.	Permisos, aprobación de pagos, cupos finales o alcance de gimnasio.
Router y stores	Ordenar navegación y mantener estado temporal.	Autorizar una llamada directa a la API.
Backend PHP	Validar, autorizar, aplicar reglas y coordinar transacciones.	Confiar sin verificación en IDs o estados enviados por el cliente.
MySQL	Garantizar relaciones, unicidad y	Interpretar por sí solo la intención

Zona	Puede hacer	No debe decidir por sí sola
	persistencia transaccional.	de negocio.
Proveedores externos	Informar resultados de su dominio, como un pago.	Crear una sesión o decidir permisos de GymTrack.

El navegador se considera un entorno controlado por el usuario. Por eso un botón oculto no equivale a una prohibición y un parámetro como `gym_id` nunca basta para demostrar que alguien pertenece a ese gimnasio.

9.5 Ejemplo interno: reservar una clase

1. `MemberScheduleView` solicita las sesiones disponibles para el gimnasio activo.
2. El store de agenda conserva la lista, la carga y el error visible.
3. Al reservar, el cliente envía el ID de sesión y una clave idempotente.
4. El router PHP encuentra el método de `ScheduleController`.
5. El middleware reconstruye la sesión desde la cookie, exige correo verificado y comprueba `classes.read` o el permiso correspondiente.
6. El controlador valida que el ID sea válido y obtiene al socio desde la sesión; no acepta que el navegador elija otro usuario.
7. `ScheduleRepository` inicia una transacción y bloquea la sesión fechada.
8. Se verifica gimnasio, estado, horario, membresía, reserva previa y capacidad.
9. Si hay cupo, se crea la reserva confirmada; si no lo hay y la política lo permite, se incorpora a espera.
10. Se registra un evento, se confirma la transacción y se devuelve el estado actual.
11. Vue actualiza la tarjeta y muestra éxito únicamente con la respuesta del servidor.

Este recorrido explica por qué una funcionalidad aparentemente simple involucra varias capas: el diseño ayuda a usarla, PHP protege la regla y MySQL evita que dos solicitudes simultáneas rompan el cupo.

9.6 Manejo de fallos y observabilidad

Cada respuesta de API incluye o relaciona un `request_id`. El frontend puede mostrar un mensaje comprensible mientras el servidor conserva un identificador útil para investigar. En producción no se envían trazas internas al navegador. Los eventos de seguridad, auditoría administrativa, pagos, reservas y notificaciones mantienen historiales separados porque responden a preguntas distintas.

Las operaciones externas se diseñan para tolerar retrasos y reintentos. Un webhook puede llegar más de una vez; un correo puede fallar temporalmente; una persona puede presionar dos veces un botón. Las claves idempotentes y los registros de eventos permiten reconocer que se trata de la misma intención.

10 · Frontend

10.1 Estructura

```
frontend/src/  
├─ assets/      marca, fotografías y recursos  
├─ components/  administración, socio, avisos, público y UI base  
├─ composables/ comportamiento reutilizable como foco modal  
├─ layouts/     shell administrativo  
├─ router/      rutas y guards  
├─ services/    cliente de API  
├─ stores/      estado Pinia por dominio  
├─ styles/      tokens y estilos base  
├─ utils/       geolocalización y utilidades  
└─ views/       pantallas públicas, de socio y administrativas
```

10.2 Navegación

La aplicación tiene rutas públicas, rutas autenticadas y una rama `/administracion`. Los guards del frontend mejoran la experiencia: redirigen al login, verificación de correo, cambio obligatorio de contraseña o acceso denegado. No sustituyen la autorización del backend.

La administración carga vistas bajo demanda y muestra destinos de acuerdo con `requiredPermission`. El socio dispone de una navegación superior y una barra inferior móvil de cinco destinos: Inicio, Agenda, Progreso, Pagos y Perfil.

10.3 Estado

Los stores separan autenticación, contexto administrativo, recursos, agenda, pagos, promociones, notificaciones, sistema y experiencia del socio. Las solicitudes cancelables evitan que respuestas antiguas reemplacen el contexto actual.

10.4 Estados de interfaz

Los componentes compartidos incluyen loading, skeleton, vacío, error, reintento, alertas, badges, toasts, botones con `loading/disabled`, diálogos, drawers y formularios con mensajes asociados. El patrón exige que una operación sólo muestre éxito después de una respuesta confirmada.

10.5 Cómo se construye una pantalla

Una vista coordina el caso de uso, pero intenta no resolverlo todo dentro del mismo archivo. Los componentes base aportan botones, campos, diálogos y mensajes consistentes. Los componentes de dominio representan piezas como navegación administrativa, filtros, tarjetas de gimnasio o avisos. Los stores se ocupan de datos compartidos y los servicios concentran la comunicación HTTP.

El ciclo habitual de una pantalla es:

1. leer parámetros de ruta y estado de sesión;
2. mostrar una estructura inicial o skeleton;
3. solicitar datos mediante el store o servicio;
4. renderizar contenido, vacío o error;
5. validar localmente lo necesario para ayudar a la persona;
6. enviar la mutación y bloquear la acción repetida mientras está en curso;
7. incorporar la respuesta confirmada o explicar el fallo junto al control correspondiente.

La validación de frontend se utiliza para mejorar la experiencia, no para proteger datos.

Aunque el navegador marque un correo o una fecha como válidos, PHP vuelve a comprobarlos.

10.6 Contexto, cancelación y datos obsoletos

Cambiar de gimnasio altera casi toda la información administrativa. Para evitar que una solicitud lenta del gimnasio anterior sobrescriba el nuevo contexto, los stores cancelan o descartan respuestas obsoletas. También se limpian recursos sensibles al cerrar sesión. Esta medida responde a un problema frecuente en aplicaciones reactivas: una pantalla puede ser visualmente correcta y aun así mostrar por unos segundos datos del contexto equivocado si no controla el orden de las respuestas.

10.7 Responsive y comportamiento táctil

La interfaz no se limita a reducir tamaños. En escritorio, administración usa sidebar y el explorador geográfico distribuye mapa y panel. En móvil, la navegación prioriza destinos frecuentes, los controles táctiles aumentan su área útil y el mapa utiliza un bottom sheet con posiciones cerrada, media y completa. Los diálogos conservan foco y bloquean el scroll del fondo. La sección [Responsive y accesibilidad](#) describe los anchos verificados.

11 · Backend y API

11.1 Organización

```
backend/
├─ config/           conexión y configuración
├─ controllers/      entrada HTTP y validación de caso de uso
├─ middleware/       sesión, correo, rol, permisos y CSRF
├─ models/           repositorios y modelos de persistencia
├─ routes/           mapa de endpoints
├─ seeders/          dataset de presentación
├─ services/         seguridad, integraciones y utilidades de dominio
├─ console.php       comandos operativos
└─ index.php         punto de entrada de la API
```

11.2 Convenciones de API

- respuestas JSON UTF-8;
- códigos HTTP para validación, autenticación, permisos, conflicto y errores;
- `request_id` validado o generado por el backend;
- mutaciones autenticadas con CSRF;
- `Idempotency-Key` en operaciones susceptibles a repetición;
- paginación y filtros para listados administrativos;
- descargas con autorización y cabeceras de cache privadas;
- errores detallados sólo en desarrollo.

11.3 Familias de endpoints

Prefijo	Responsabilidad
<code>/api/auth/*</code>	registro, login, correo, recuperación y contraseña.

Prefijo	Responsabilidad
<code>/api/me/*</code>	sesión, contexto, avisos y preferencias.
<code>/api/public/*</code>	catálogo, ficha, planes y archivos públicos.
<code>/api/member/*</code>	perfil, favoritos, progreso, medidas y carné.
<code>/api/class-sessions,</code> <code>/api/bookings</code>	agenda y reservas del socio.
<code>/api/payments/*</code>	pagos del socio y checkout.
<code>/api/admin/*</code>	administración, operación, finanzas, exportaciones y promociones.
<code>/api/webhooks/mercado-pago</code>	confirmación externa de pagos.

Existen endpoints heredados de clases, reservas y membresías que se conservan por compatibilidad. La experiencia administrativa actual utiliza los contratos modernos con contexto de gimnasio y auditoría.

11.4 Recorrido interno del backend

El punto de entrada prepara cabeceras, carga clases y establece el tratamiento seguro de excepciones. El router normaliza el método y la URL y busca una coincidencia estática o dinámica. El controlador no debería contener consultas extensas: interpreta HTTP, valida el caso de uso y delega. Los repositorios conocen SQL y los servicios encapsulan capacidades transversales o externas.

Pieza	Pregunta que responde	Ejemplo
Router	¿Qué código atiende esta URL y método?	<code>POST /api/payments/checkout</code> → <code>PaymentController</code> .
Middleware	¿La solicitud puede continuar?	Sesión, CSRF, correo, rol, permiso y contexto.
Controlador	¿Los datos representan una petición válida?	Monto, IDs, transición o archivo aceptable.
Servicio	¿Cómo se ejecuta?	Servicios internos o externos.

Pieza	Pregunta que responde	Ejemplo
Repositorio	¿Cómo se lee o modifica el dominio en MySQL?	Bloquear cupo, listar finanzas o guardar un evento.
Responder	¿Cómo se devuelve el resultado de forma uniforme?	Código HTTP, JSON y <code>request_id</code> .

11.5 Códigos y respuestas

Los códigos de estado forman parte del contrato. Un `400` indica una petición mal formada; `401`, ausencia o vencimiento de sesión; `403`, falta de autorización; `404`, recurso inexistente dentro del alcance; `409`, conflicto como duplicado o transición incompatible; `419`, CSRF vencido; `422`, validación de campos; `429`, límite de intentos; y `500`, un error interno que no debe revelar detalles productivos.

El frontend traduce estos resultados a mensajes accionables. Por ejemplo, un `419` invita a renovar la sesión de seguridad y reintentar, mientras que un `403` no debería sugerir que un simple reintento resolverá la falta de permiso.

11.6 Operaciones de consola

`backend/console.php` concentra acciones que no pertenecen a una petición web: aplicar y revisar migraciones, comprobar esquema, administrar el dataset demo, regenerar credenciales de presentación y procesar notificaciones pendientes. Separar estos comandos evita exponer tareas operativas como endpoints públicos.

El seeder de demostración es deliberadamente condicionado. Antes de escribir datos comprueba entorno y banderas, y en producción exige una excepción explícita que por defecto está desactivada. Sus operaciones de estado, reseteo y eliminación permiten preparar una presentación sin mezclarla con información real.

12 · Base de datos

12.1 Estrategia

MySQL 8 es la fuente de verdad. El repositorio contiene un esquema inicial y nueve migraciones versionadas, cada una con reversión controlada cuando corresponde. Los identificadores históricos de migración se conservan para no romper bases existentes, Docker ni `schema-check`.

Las relaciones usan claves foráneas, índices, restricciones únicas y estados explícitos. Los flujos de cupos, pagos, reembolsos y cambios sensibles emplean transacciones, locks o claves idempotentes.

12.2 Inventario lógico de tablas

El esquema actual crea 49 tablas, agrupadas de la siguiente forma:

Dominio	Tablas
Identidad y seguridad	<code>usuarios</code> , <code>roles</code> , <code>permisos</code> , <code>rol_permisos</code> , <code>user_sessions</code> , <code>email_verification_tokens</code> , <code>password_reset_tokens</code> , <code>user_consent</code> s, <code>rate_limit_attempts</code> , <code>security_events</code> , <code>role_migration_report</code>
Gimnasios y alcance	<code>gimnasios</code> , <code>gimnasio_sedes</code> , <code>usuario_gimnasio_roles</code> , <code>usuario_gimnasio_permisos</code> , <code>owner_registration_requests</code> , <code>invitaciones_gimnasio</code>
Personas	<code>socio_perfiles</code> , <code>empleado_perfiles</code> , <code>entrenador_perfiles</code> , <code>entrenador_gimnasios</code>
Membresías	<code>planes_membresia</code> , <code>membresias</code> , <code>membresia_historial</code>
Agenda	<code>actividades</code> , <code>clases</code> , <code>sesiones_clase</code> , <code>reservas</code> , <code>reserva_eventos</code> , <code>asistencias</code> , <code>calificaciones</code>
Pagos y reportes	<code>pagos</code> , <code>pago_eventos</code> , <code>pago_reembolsos</code> , <code>exports</code>
Promociones y avisos	<code>promociones</code> , <code>promocion_gimnasios</code> , <code>notificaciones</code> , <code>notificacion_envios</code> , <code>notificacion_preferencias</code> , <code>marketing_unsubscribe_tokens</code>
Experiencia del socio	<code>socio_preferencias</code> , <code>socio_gimnasios_favoritos</code> , <code>socio_actividades_favoritas</code> , <code>socio_mediciones</code>

Dominio	Tablas
Archivos y trazabilidad	<code>archivos</code> , <code>audit_logs</code> , <code>demo_datasets</code> , <code>schema_migrations</code>

12.3 Relaciones centrales

- un usuario tiene un rol global y puede tener roles por gimnasio;
- un gimnasio tiene sedes, personas, planes, clases, sesiones, pagos y promociones;
- una clase recurrente produce sesiones fechadas;
- una reserva pertenece a un socio y a una sesión, y puede generar eventos y asistencia;
- una membresía relaciona socio, gimnasio y versión de plan;
- un pago se relaciona con membresía, gimnasio, eventos y posibles reembolsos;
- una promoción puede abarcar varios gimnasios y generar entregas por canal;
- todo registro demo sensible conserva `is_demo` y `demo_dataset_id`.

12.4 Del modelo inicial al modelo actual

El documento original describía diez tablas normalizadas y detallaba `usuarios`, `membresias`, `clases` y `reservas`. Esa propuesta sigue siendo reconocible, pero el sistema real necesitó separar conceptos que al principio parecían equivalentes. Una clase como “Funcional los lunes” no es la misma entidad que la sesión del 31 de agosto a las 18:00; un entrenador puede trabajar en varios gimnasios; una membresía necesita historial; y un pago requiere eventos y reembolsos.

La expansión a 49 tablas no busca complejidad por sí misma. Cada separación responde a una necesidad de integridad, trazabilidad o reutilización. El modelo evita guardar listas importantes dentro de un único texto y prefiere relaciones explícitas que puedan filtrarse por gimnasio.

12.5 Migraciones y compatibilidad

`gymtrack_database.sql` proporciona la base histórica. Las migraciones agregan cambios incrementales y `schema_migrations` registra cuáles ya se aplicaron. Los nombres históricos que contienen referencias a etapas del desarrollo se mantienen porque forman parte del contrato operativo de bases existentes; renombrarlos podría provocar que un entorno intente aplicar el mismo cambio dos veces.

Una migración debe ser repetible en el sentido operativo: antes de crear una estructura comprueba su existencia y, cuando dispone de archivo de reversión, ese rollback se limita a los objetos introducidos por la versión correspondiente. Antes del arranque productivo, `schema-check` valida que la base haya alcanzado todas las versiones esperadas.

12.6 Transacciones e invariantes

Las transacciones se utilizan cuando una acción comprende varias escrituras que deben ocurrir juntas. En una reserva no basta con insertar una fila: también puede cambiar la disponibilidad, generarse un evento y modificarse la lista de espera. En un pago, el estado, el evento y la activación de membresía no deberían quedar a medias.

Las restricciones únicas funcionan como una última defensa ante concurrencia. Aunque PHP consulte primero si existe una reserva, dos solicitudes podrían hacer esa consulta al mismo tiempo. La combinación de lock, transacción, índice único y manejo de conflicto protege el resultado incluso bajo esa carrera.

12.7 Aislamiento por gimnasio y dataset

La mayoría de los datos operativos incluye `gimnasio_id`; los datos de presentación incluyen además su marca demo. Los repositorios no se limitan al ID del recurso solicitado: combinan recurso, gimnasio activo y dataset de sesión. Esto reduce el riesgo de que una URL con un ID válido de otro gimnasio revele o modifique información ajena.

13 · Seguridad

13.1 Autenticación y sesiones

- contraseñas mediante `password_hash` con BCrypt y coste 12;
- rehash automático cuando cambia la configuración;
- cookie `HttpOnly`, `SameSite=Lax` y `Secure` obligatorio en producción;
- ID de sesión regenerado al iniciar y al cambiar contexto;
- expiración por inactividad y límite absoluto;
- sesiones registradas en MySQL y revocables individualmente o en conjunto;
- tokens de correo, recuperación e invitación guardados como hash y de un solo uso.

13.2 Autorización

PHP valida correo, rol, permiso y gimnasio activo. Los repositorios vuelven a aplicar el alcance del dataset y del gimnasio para reducir riesgos IDOR. El administrador general accede sólo al conjunto real o demo que corresponde a su sesión.

13.3 Protección de solicitudes

- CSRF en POST, PUT, PATCH y DELETE autenticados;
- CORS limitado al origen configurado;
- CSP restrictiva para respuestas API;
- `X-Content-Type-Options`, `Referrer-Policy` y `Permissions-Policy`;
- rate limiting persistido con actualización atómica;
- validación de UUID para `request_id` e idempotencia;
- errores seguros y correlación por solicitud.

13.4 Datos, archivos e integraciones

- PDO con consultas preparadas;
- validación de MIME, tamaño y dimensiones;
- almacenamiento privado fuera del webroot y claves sin traversal;
- archivos con permisos restrictivos;
- webhooks de Mercado Pago con firma HMAC y consulta directa al proveedor;
- referencias externas únicas e historial de eventos;
- auditoría administrativa y eventos de seguridad;

- secretos exclusivamente mediante entorno o secretos de Docker.

13.5 Modelo práctico de amenazas

La seguridad se diseñó alrededor de riesgos concretos:

Riesgo	Control principal	Control complementario
Robo o fijación de sesión	Regeneración de ID y cookie segura	Expiración, registro y revocación.
Solicitud enviada desde otro sitio	Token CSRF	<code>SameSite</code> , CORS y origen configurado.
Inyección SQL	PDO y parámetros preparados	Validación de tipos y consultas acotadas.
Acceso horizontal a otro gimnasio	Contexto de sesión y autorización PHP	Filtros tenant repetidos en repositorios.
Fuerza bruta	Rate limit persistente	Eventos de seguridad y mensajes neutros.
Archivo malicioso	MIME, tamaño, dimensiones y almacenamiento privado	Descarga autorizada y nombre interno generado.
Pago falsamente aprobado	Webhook firmado y consulta al proveedor	Referencia única, idempotencia e historial.
Exposición de secretos	Variables de entorno	Preflight y exclusión de Git.
Duplicación por reintento	Clave idempotente	Restricciones únicas y eventos.

13.6 Ciclo de una sesión

Al iniciar sesión, el backend verifica la contraseña con el hash guardado, regenera el identificador, registra la sesión y entrega una cookie que JavaScript no puede leer. `GET /api/me` reconstruye usuario, rol, permisos, gimnasios disponibles y contexto. Las mutaciones requieren además un token CSRF asociado a la sesión.

Cuando la persona cambia de gimnasio, el servidor valida que tenga asociación, cambia el contexto y rota los datos de seguridad. Al cerrar sesión, invalida tanto la sesión PHP como su registro persistente. El cierre global permite revocar otros dispositivos sin conocer sus cookies.

13.7 Privacidad y minimización

La geolocalización se usa para calcular distancia en el navegador y no se persiste automáticamente. El carné digital no incorpora datos personales en texto abierto: utiliza un token firmado y de duración limitada. Las medidas corporales son opcionales; el IMC se presenta como orientación no diagnóstica. Las preferencias de marketing se separan de las comunicaciones transaccionales necesarias.

La versión de presentación utiliza personas y gimnasios ficticios. Ningún dato demo debería copiarse a producción ni presentarse como si perteneciera a un comercio real.

14 · Roles y permisos

14.1 Roles del sistema

Rol	Alcance real
Socio	Perfil, gimnasios autorizados, clases, reservas, membresías y pagos propios.
Empleado	Operación del gimnasio según permisos base y overrides.
Dueño	Acceso amplio a los gimnasios asociados, configuración y resultados.
Administrador general	Permisos globales y modo soporte auditado dentro del dataset correspondiente.

14.2 Permisos modernos

El sistema define, entre otros: `profile.read`, `profile.write`, `gyms.read`, `members.read`, `members.write`, `classes.read`, `classes.write`, `reservations.read`, `reservations.write`, `attendance.write`, `memberships.read`, `memberships.write`, `payments.read`, `payments.manual`, `finance.read`, `reports.export`, `staff.manage`, `gym.configure`, `promotions.read` y `promotions.write`.

Los permisos de un empleado pueden sobrescribirse por asociación de gimnasio. Que una opción no aparezca en el menú no es la única defensa: cada endpoint comprueba nuevamente el permiso.

14.3 Matriz resumida de acceso

Área	Socio	Empleado	Dueño	Admin. general
Perfil y preferencias propias	Sí	Sí	Sí	Sí
Agenda y reservas propias	Sí	Según asociación	Según asociación	Soporte auditado
Socios del gimnasio	No	Según permiso	Sí	Soporte auditado
Empleados y entrenadores	No	Según permiso	Sí	Soporte auditado
Clases y asistencia	Lectura/reserva	Según permiso	Sí	Soporte auditado
Membresías y pagos	Propios	Según permiso	Sí	Soporte auditado
Finanzas y reportes	No	Según permiso	Sí	Soporte auditado
Configuración del gimnasio	No	Según permiso explícito	Sí	Soporte auditado

“Soporte auditado” significa que el administrador general no opera fuera de todo contexto: selecciona el gimnasio y las acciones sensibles dejan registro. El dueño asociado a varios gimnasios puede cambiar de contexto; la API vuelve a resolver sus permisos en cada cambio.

15 · Funcionalidades

15.1 Experiencia pública

- página de inicio y presentación del producto;
- catálogo de gimnasios desde MySQL;
- búsqueda y filtros por ciudad, categoría, servicio y estado;
- mapa sincronizado con las tarjetas;
- ficha pública con sedes, horarios, contacto, servicios y planes;
- registro de socio y solicitud de cuenta de dueño;
- páginas informativas de privacidad, términos, accesibilidad y contacto.

15.2 Identidad

- registro con aceptación de términos y privacidad;
- Turnstile configurable;
- verificación y reenvío de correo;
- login, logout y cierre global;
- recuperación, restablecimiento y cambio de contraseña;
- listado y revocación de sesiones;
- invitaciones con vencimiento y un solo uso.

15.3 Operación

- contexto de gimnasio activo;
- resumen y actividad administrativa;
- gestión de gimnasios y sedes;
- socios, empleados y entrenadores;
- planes versionados y membresías;
- clases, sesiones, cupos, reservas, espera y asistencia;
- pagos manuales, checkout, reembolso, finanzas y reportes;
- promociones, resultados y notificaciones;
- archivos e historial administrativo.

15.4 Socio

- resumen personal y próxima clase;
- objetivo y progreso mensual;

- perfil con fotografía;
- preferencias de horarios y avisos;
- favoritos de gimnasios y actividades;
- historial de asistencia, membresías y pagos;
- medidas opcionales e IMC orientativo;
- carné digital y verificación administrativa.

16 · Flujos de uso

16.1 Visitante

1. Abre el catálogo.
2. Filtra gimnasios o solicita ubicación.
3. Selecciona una tarjeta o marcador.
4. Consulta la ficha y los planes.
5. Se registra como socio o solicita cuenta de dueño.

16.2 Socio

1. Inicia sesión y verifica su correo.
2. Elige un gimnasio asociado.
3. Revisa resumen, membresía y deuda.
4. Reserva una sesión con cupo o entra en lista de espera.
5. Añade el evento a Google Calendar o descarga ICS.
6. Consulta progreso, pagos, favoritos y carné.

16.3 Personal administrativo

1. Inicia sesión y selecciona el gimnasio.
2. La navegación se adapta a sus permisos.
3. Consulta resumen y actividad.
4. Gestiona el módulo autorizado.
5. Toda mutación se valida, audita y mantiene dentro del gimnasio activo.

16.4 Pago con proveedor

1. El socio solicita checkout con un `Idempotency-Key`.

2. PHP crea la referencia externa y pide una preferencia a Mercado Pago.
3. El socio paga fuera de GymTrack.
4. La URL de regreso sólo informa que el proceso continúa.
5. El webhook firmado llega al backend.
6. PHP consulta el pago directamente al proveedor.
7. Sólo entonces persiste el estado y activa la membresía si corresponde.

16.5 Recuperación de acceso

1. La persona ingresa su correo en `/recuperar`.
2. La respuesta es neutra para no revelar si existe una cuenta.
3. Si corresponde, PHP genera un token aleatorio, guarda únicamente su hash y envía un enlace con vencimiento.
4. La vista `/restablecer/:token` solicita la nueva contraseña.
5. El backend comprueba token, vencimiento y uso previo, actualiza el hash e invalida el token.
6. Las sesiones anteriores pueden revocarse para reducir el riesgo de una cuenta comprometida.

El correo transporta una capacidad temporal; no contiene la contraseña ni permite reutilizar el token después del cambio.

16.6 Crear y publicar una promoción

1. El dueño o empleado autorizado abre Promociones dentro del gimnasio activo.
2. Define contenido, audiencia, período, zona horaria, descuento, canales y sedes.
3. El frontend valida campos para ayudar; PHP comprueba permisos, fechas, gimnasios y transición.
4. La promoción queda como borrador o programada y conserva una versión.
5. El proceso de notificaciones genera entregas identificables por persona, canal y versión.
6. Antes de enviar, se revisan consentimiento y preferencias.
7. Los resultados distinguen pendiente, enviado, fallido y omitido.

Pausar una promoción detiene entregas nuevas sin borrar su historial. Finalizarla cierra el ciclo y evita presentar una campaña pasada como activa.

16.7 Cambiar el gimnasio de trabajo

1. La persona abre el selector de contexto.
2. El frontend muestra únicamente gimnasios recibidos desde la sesión.
3. PHP confirma nuevamente la asociación y el dataset.
4. El servidor cambia el gimnasio activo y rota sesión/CSRF.
5. Los stores administrativos limpian datos del contexto anterior.
6. Cada módulo recarga métricas y recursos con el nuevo alcance.

Este flujo es clave para dueños con varios establecimientos. El cambio no altera datos por sí mismo; modifica la frontera dentro de la cual se permiten las siguientes operaciones.

17 · Administración

La sección visible **Administración** contiene:

- Resumen.
- Gestión operativa.
- Socios.
- Verificar carné.
- Empleados.
- Entrenadores.
- Clases.
- Reservas.
- Membresías.
- Pagos.
- Promociones.
- Finanzas.
- Reportes.
- Configuración.

El resumen ofrece accesos rápidos, métricas y actividad. Finanzas y reportes están disponibles directamente desde la navegación cuando el usuario tiene `finance.read` y `reports.export`.

La configuración permite alta, edición y archivado lógico de gimnasios, además de sedes. El archivado evita destruir historial. Las personas se incorporan mediante asociación o invitación; los entrenadores tienen perfil y gimnasios asignados.

17.1 Una jornada operativa posible

Al comenzar el día, el personal puede revisar el resumen y las sesiones próximas. Desde Reservas comprueba cupos y lista de espera; desde Verificar carné confirma identidad y membresía sin leer datos personales desde el QR. Durante la jornada marca asistencias, registra una cuota manual si tiene permiso y atiende invitaciones o cambios de membresía.

El dueño puede alternar entre gimnasios, revisar ingresos, deuda, promociones y reportes. Los accesos rápidos reducen pasos para tareas frecuentes, mientras que acciones destructivas o económicas requieren confirmación, permiso y auditoría.

17.2 Estados administrativos

Los listados administrativos combinan filtros, búsqueda, orden y paginación. Un vacío explica si todavía no existen registros o si los filtros no encontraron coincidencias. Las mutaciones muestran carga y desactivan repetición. Los errores de validación se ubican junto al campo; los conflictos explican si el dato cambió o ya existe.

El archivado se prefiere a borrar cuando existe historial relacionado. Así, una sede cerrada o un gimnasio inactivo deja de ofrecerse sin romper pagos, reservas o auditorías pasadas.

18 · Experiencia del socio

El panel personal combina información del gimnasio activo:

- próxima reserva;
- progreso mensual calculado desde asistencias;
- membresía vigente o estado pendiente;
- pago pendiente;
- promociones aplicables por audiencia;
- acciones rápidas a agenda, pagos, progreso, perfil y carné.

El perfil admite nombre, apellido, teléfono, fecha de nacimiento y fotografía. Las preferencias guardan objetivo mensual, visibilidad del IMC orientativo y franjas horarias. Los favoritos se persisten en MySQL, no sólo en el navegador.

Las mediciones de altura y peso son opcionales. El IMC se presenta como orientación y no como diagnóstico médico. El carné genera un QR con token firmado de corta duración y sin datos personales. El verificador administrativo vuelve a validar gimnasio, dataset, socio y membresía en PHP.

19 · Pagos, finanzas y reportes

19.1 Pagos

Los estados internos contemplan pendiente, aprobado, rechazado, vencido, reembolsado y cancelado. El repositorio conserva referencia externa, proveedor, modo, importes, moneda, método, eventos y claves idempotentes.

El adaptador inicial es Mercado Pago y cumple la interfaz `PaymentProviderInterface`. La aplicación puede operar sin credenciales para consulta y pagos manuales, pero no ofrece checkout funcional hasta que el proveedor esté configurado.

19.2 Finanzas

La vista financiera obtiene desde MySQL:

- ingresos del mes y de seis meses;
- comparación con el período anterior;
- pagos por estado;
- deuda total;
- agrupación por gimnasio, plan y método;
- transacciones con filtros, orden y paginación.

19.3 Reportes

Los reportes se generan para pagos o finanzas. El backend crea un registro de exportación, produce el archivo, registra tamaño y MIME, limita su vigencia y exige el mismo gimnasio para descargarlo.

- **XLSX:** contiene el detalle completo en una hoja compatible con Excel.
- **PDF:** resumen textual de una página; limita la vista a 45 filas e indica que Excel debe usarse para el detalle completo.

20 · Promociones, notificaciones y calendario

20.1 Promociones

Una promoción contiene nombre, descripción, audiencia, inicio, fin, zona horaria, descuento, código, imagen, canales, gimnasios y versión. Sus transiciones reales son borrador, programada o activa, pausada y finalizada. La eliminación es lógica.

Las audiencias incluyen todos los socios, socios activos, socios con deuda y socios inactivos. Antes de abarcar otro gimnasio, PHP comprueba que el actor pueda acceder y publicar en él.

20.2 Notificaciones

El centro de notificaciones permite listar, marcar una o todas como leídas y conservar un total no leído. Las preferencias separan avisos transaccionales y marketing por canal.

El dispatcher reclama entregas, comprueba consentimiento y preferencias, evita duplicados mediante claves idempotentes, registra enviados, fallidos u omitidos y reintenta fallos recuperables. La baja publicitaria usa un token y no desactiva mensajes transaccionales necesarios.

20.3 Calendario

Cada reserva confirmada de una sesión programada puede producir:

- una URL de Google Calendar con título, fecha, hora, zona, dirección, entrenador y descripción;
- un archivo ICS con UID estable y secuencia de versión.

La sincronización automática, actualización remota y cancelación mediante OAuth no están implementadas. La interfaz debe decir que el evento es una copia manual.

21 · Mapa y geolocalización

El explorador usa Leaflet y tiles de OpenStreetMap. Los datos de gimnasios provienen de la API pública y MySQL.

21.1 Funciones verificadas

- solicitud de ubicación con consentimiento;

- estados idle, solicitando, listo, rechazado, no disponible, no compatible y fallback;
- vista predeterminada centrada en Uruguay;
- distancia Haversine calculada en el navegador;
- orden por cercanía sólo cuando existe ubicación;
- filtros y búsqueda;
- agrupación por celdas proyectadas de 72 px;
- selección sincronizada entre marcador y tarjeta;
- recentrado en usuario o conjunto de gimnasios;
- carga, error y reintento de mapa y catálogo.

21.2 Panel geográfico guardado

En escritorio el panel de gimnasios se contrae y libera espacio al mapa. En móvil funciona como bottom sheet cerrado, medio o completo y admite arrastre. Las preferencias del panel, posición del sheet y filtros se guardan en `localStorage` bajo `gymtrack.explorer.preferences.v1`.

Los favoritos son distintos: un socio autenticado los guarda en MySQL. La ubicación personal nunca se envía a la API ni se persiste.

22 · Sistema de demostración

22.1 Principios

El dataset demo se crea sólo con `APP_ENV=demo` o `SEED_DEMO_DATA=true`. El seeder exige contraseña por entorno, bloquea producción por defecto y puede ejecutarse repetidamente sin duplicar registros.

Comandos disponibles:

```
php console.php seed:demo
php console.php seed:demo --reset
php console.php seed:demo --status
php console.php seed:demo --remove
```

22.2 Gimnasios ficticios

Gimnasio	Ciudad	Escenario
GymTrack Centro	Montevideo	Tres clases, sesiones, reservas, pagos y

Gimnasio	Ciudad	Escenario
		promoción.
Norte Fitness Club	Salto	Estado vacío sin clases.
Titan Training	Maldonado	Membresía activa del socio demo.
Punto Activo	Canelones	Cerrado temporalmente.
Arena Functional Gym Ciudad de la Costa		Plan y escenario promocional; WhatsApp pendiente.

Todos usan direcciones, teléfonos y correos ficticios, `America/Montevideo`, imágenes locales y marcas `is_demo/demo_dataset_id`.

22.3 Cuentas

Rol	Correo	Asociaciones demo actuales
Socio	<code>socio.demo@gymtrack.local</code>	GymTrack Centro y Titan Training.
Empleado	<code>empleado.demo@gymtrack.local</code>	GymTrack Centro.
Dueño	<code>dueno.demo@gymtrack.local</code>	GymTrack Centro y Arena Functional Gym.
Admin general	<code>admin.demo@gymtrack.local</code>	Los cinco gimnasios del dataset.

La contraseña es el valor local de `DEMO_USER_PASSWORD`; no existe una contraseña fija en el repositorio. La etiqueta “Datos de demostración” aparece cuando el dataset está activo o la cuenta es demo.

23 · Diseño e identidad visual

La dirección visual comprobada en [DESIGN.md](#) y los tokens del frontend se resume como **“Calma operativa con energía física”**. La interfaz usa fondo carbón, superficies mate, azul reservado para acciones y colores semánticos sólo cuando existe un estado real.

Principios:

- una acción principal por contexto;
- densidad informativa con jerarquía y espacio consistente;
- gimnasio, dataset y rol visibles cuando condicionan los datos;
- color como apoyo, nunca como única señal;
- confirmaciones únicamente después de persistencia real;
- fotografías en la experiencia pública y sobriedad en la operación.

Paleta GymTrack

Tokens reales del sistema visual

Fondo sutil
#10151B

Superficie 1
#141A21

Superficie 2
#19212B

Superficie elevada
#202A35

Acción
#0969DA

Información
#68B6FF

Foco
#79BAFF

Superficie inversa
#F3F6F8

Éxito
#43C17A

Advertencia
#F3AD4E

Peligro
#F36B6B

Borde sutil
#27313D

Texto primario · #F4F7F9

Texto secundario · #AEB9C5

Texto terciario · #7F8B98



24 · Paleta de colores

24.1 Colores principales del sistema

Token / nombre	HEX	RGB	Uso
Fondo base	#0B0E12	11, 14, 18	Lienzo general.
Fondo sutil	#10151B	16, 21, 27	Campos y zonas hundidas.
Superficie 1	#141A21	20, 26, 33	Tarjetas y paneles.
Superficie 2	#19212B	25, 33, 43	Selección y controles secundarios.
Superficie elevada	#202A35	32, 42, 53	Capas activas.
Superficie inversa	#F3F6F8	243, 246, 248	Carné y QR.
Borde sutil	#27313D	39, 49, 61	Separación cotidiana.
Borde fuerte	#394757	57, 71, 87	Controles y estados activos.
Texto primario	#F4F7F9	244, 247, 249	Títulos y contenido principal.
Texto secundario	#AEB9C5	174, 185, 197	Explicación.
Texto terciario	#7F8B98	127, 139, 152	Metadatos.
Acción	#0969DA	9, 105, 218	Acción principal.
Acción hover	#075FC5	7, 95, 197	Respuesta hover.
Acción presionada	#0553AE	5, 83, 174	Respuesta active.
Foco	#79BAFF	121, 186, 255	Anillo de teclado.
Información	#68B6FF	104, 182, 255	Marca, enlaces e información.
Éxito	#43C17A	67, 193, 122	Estado confirmado.
Advertencia	#F3AD4E	243, 173, 78	Atención o condición.
Peligro	#F36B6B	243, 107, 107	Error o acción destructiva.

24.2 Colores del símbolo

El isotipo añade `#0D0D0D`, blanco, gris metálico `#C8C8C8` y el gradiente azul formado por `#38BDF8`, `#0077FF` y `#00BFFF`. Estos colores pertenecen a la marca; no reemplazan los tokens funcionales de la interfaz.

25 · Tipografía

La familia real es **Manrope Variable**, con Manrope y fuentes sans-serif del sistema como fallback.

Uso	Peso	Tamaño/ritmo real
Display	730	<code>clamp(2.5rem, 7vw, 5.5rem)</code> , línea 1.08.
Headline	730	<code>clamp(2rem, 4vw, 3.6rem)</code> , línea 1.08.
Título	730	<code>clamp(1.15rem, 2vw, 1.45rem)</code> , línea 1.08.
Cuerpo	400	<code>1rem</code> , línea 1.6.
Etiqueta	660	<code>0.875rem</code> , línea 1.2.
Marca	780	aproximado Tracking negativo y <code>Track</code> en celeste informativo.

Los títulos usan tracking `-0.035em`. El texto continuo se limita visualmente a unas 60–70 letras por línea cuando la composición lo permite.

26 · Sistema de logos

Todas las construcciones derivan del símbolo `gymtrack-mark.svg` existente y del wordmark Manrope. No representan marcas diferentes.

26.1 Logotipo

Versión tipográfica basada en el nombre, sin símbolo.

GymTrack

Archivo: `logotipo-gymtrack.svg`.

26.2 Isotipo

Símbolo independiente para espacios compactos.



Archivo: [isotipo-gymtrack.svg](#).

26.3 Imagotipo

Símbolo y nombre alineados, visualmente relacionados pero separables.



Archivo: [imagotipo-gymtrack.svg](#).

26.4 Isologo

Composición cerrada en la que símbolo y nombre funcionan como una unidad dentro de una misma superficie.



Archivo: [isologo-gymtrack.svg](#).

26.5 Variantes disponibles

Variante	Uso recomendado	Archivo
Principal	Portadas y presentación	logo-gymtrack-principal.svg/.png
Horizontal	Cabeceras anchas	logo-gymtrack-horizontal.svg
Vertical	Piezas estrechas o centradas	logo-gymtrack-vertical.svg
Compacta	Avatares y espacios reducidos	logo-gymtrack-compacto.svg
Fondo claro	Documentos claros	logo-gymtrack-fondo-claro.svg
Fondo oscuro	Interfaz principal	logo-gymtrack-fondo-oscuro.svg
Monocromática	Impresión a una tinta	logo-gymtrack-monocromatico.svg
Icono de aplicación	Icono de gran resolución	icono-aplicacion-gymtrack.svg/.png

Variante	Uso recomendado	Archivo
Favicon	Navegador	<code>frontend/public/favicon.ico</code> existente

26.6 Reducción y área de seguridad

- Isotipo digital recomendado: mínimo 32 × 32 px; preferible 44 × 44 px en controles táctiles.
- Imagotipo horizontal: no usar por debajo de 160 px de ancho; debajo de ese valor, usar isotipo.
- Área de seguridad: conservar alrededor de la composición al menos una cuarta parte del ancho del isotipo.
- No recortar la base negra ni los extremos del símbolo.

26.7 Usos correctos

- elegir la variante según el fondo;
- conservar proporciones;
- usar los colores de marca documentados;
- mantener contraste y espacio de seguridad;
- usar SVG para interfaz y PNG sólo cuando el destino no admita vector.

26.8 Usos incorrectos

- no deformar ni inclinar;
- no recolorar con tonos ajenos;
- no separar las piezas internas del isotipo;
- no agregar sombras, contornos o brillos nuevos;
- no colocar la variante oscura sobre un fondo sin contraste;
- no reemplazar Manrope por una tipografía decorativa.

27 · Responsive y accesibilidad

27.1 Responsive

Los breakpoints estructurales reales son 768 px y 1024 px. Las verificaciones del diseño contemplan 360, 390, 768, 1024, 1440 y 1920 px.

- el contenido usa un contenedor máximo de 77.5 rem;
- administración cambia sidebar por drawer bajo 1024 px;
- el socio usa barra inferior en móvil;
- tablas densas pueden transformarse en tarjetas;
- filtros y acciones se apilan;
- el mapa usa panel contraíble o bottom sheet;
- el `body` evita scroll horizontal y considera áreas seguras.

27.2 Accesibilidad implementada

- foco visible de 3 px;
- objetivos interactivos mínimos de 44 px en los componentes principales;
- labels persistentes y mensajes asociados;
- navegación por teclado en menús, diálogos, drawers y mapa;
- `aria-pressed`, `aria-expanded`, roles y textos de estado donde corresponde;
- restauración de foco y bloqueo de scroll en overlays;
- soporte de `prefers-reduced-motion` y `prefers-reduced-transparency`;
- pruebas Axe en Playwright.

Esto no constituye una certificación WCAG formal. La aplicación implementa medidas concretas y pruebas automatizadas, pero una auditoría manual completa con tecnologías de asistencia sigue siendo recomendable.

28 · Pruebas y control de calidad

28.1 Cobertura presente

- **Vitest:** 12 archivos y 32 casos declarados sobre UI, geolocalización y stores.
- **Playwright:** 8 archivos y 26 casos declarados sobre flujos públicos, autenticación, roles, administración, pagos, agenda y responsive.
- **Backend:** 19 scripts PHP/Shell de contratos, seguridad e integración.
- **CI:** auditoría npm, Vitest, build, lint PHP/Shell, contratos, preflight y construcción de imágenes.

28.2 Áreas verificadas

- autenticación, correo, sesiones y CSRF;
- integridad y aislamiento por gimnasio;
- dataset demo idempotente;
- administración y operaciones multi-gimnasio;
- clases, reservas, espera, asistencia e idempotencia;
- pagos, finanzas, reportes, reembolsos y webhook;
- mapa, distancias y catálogo;
- calendario, promociones, consentimiento y notificaciones;
- perfil, favoritos, medidas, progreso y carné;
- configuración productiva, concurrencia y códigos HTTP.

28.3 Criterio de calidad

La presencia de una prueba no reemplaza la prueba en staging de servicios externos. Mercado Pago, SMTP, Turnstile, TLS, backups y restauración deben comprobarse con credenciales y entorno equivalentes a producción.

28.4 Pirámide de comprobación

Nivel	Qué comprueba	Ventaja	Límite
Unitario	Componente, store o función aislada	Rápido y preciso para regresiones pequeñas.	No demuestra integración completa.

Nivel	Qué comprueba	Ventaja	Límite
Contrato	Rutas, esquema, cabeceras y reglas esperadas	Detecta incompatibilidades entre capas.	Puede no reproducir un navegador real.
Integración HTTP	Sesión, CSRF, permisos y persistencia	Recorre backend y MySQL.	Depende del entorno levantado.
E2E	Recorrido visible del usuario	Valida navegación e interacción real.	Es más lento y sensible a entorno.
Manual/staging	Proveedores, dispositivos y operación	Se aproxima al uso real.	Requiere procedimiento y evidencia humana.

28.5 Qué conviene probar antes de presentar

1. recrear o resetear el dataset demo sin duplicados;
2. iniciar sesión con los cuatro roles y comprobar su destino;
3. cambiar de gimnasio con el dueño y confirmar que cambien cifras y listados;
4. reservar y cancelar una sesión, incluida la lista de espera;
5. registrar un pago manual y observar su efecto financiero;
6. generar Excel y PDF y abrir ambos archivos;
7. crear, programar y pausar una promoción sin enviar duplicados;
8. negar geolocalización y confirmar que exista ubicación predeterminada;
9. revisar mapa y navegación en 360, 390, 768, 1024, 1440 y 1920 píxeles;
10. cerrar sesión y verificar que no queden datos privados visibles.

Para integraciones externas, la evidencia debe incluir la configuración usada, el identificador del evento y la respuesta real. No se considera prueba que la interfaz haya mostrado una pantalla de éxito si el proveedor no confirmó la operación.

29 · Problemas encontrados y soluciones

Esta sección no pretende mostrar un desarrollo lineal ni perfecto. GymTrack fue creciendo mientras aparecían casos que una maqueta no revela: datos simultáneos, cambios de contexto, permisos que deben comprobarse dos veces e integraciones que responden más tarde. En cada caso se documenta el problema técnico, la decisión tomada y el aprendizaje que puede servir al equipo en futuras versiones.

29.1 Datos simulados en la interfaz

Problema El catálogo y los dashboards necesitaban escenarios de presentación sin convertir Vue en la fuente de verdad.

Causa Los datos estáticos no permiten probar permisos, filtros, asociaciones ni persistencia.

Solución aplicada Se creó un seeder MySQL idempotente, condicionado por entorno, con cinco gimnasios, cuatro roles y marcas de dataset.

Resultado La presentación usa los mismos endpoints y tablas que los datos reales y puede eliminarse selectivamente.

Área involucrada Backend, base de datos y presentación.

Aprendizaje Una demostración es más convincente cuando recorre la aplicación real.

Preparar buenos datos de prueba no es “rellenar pantallas”: también sirve para descubrir asociaciones, filtros y permisos incompletos.

29.2 Protección sólo visual de rutas

Problema Ocultar una vista o usar un guard de Vue no impide llamar directamente a la API.

Causa El navegador no es una frontera de confianza.

Solución aplicada El middleware PHP verifica sesión, correo, rol, permiso y contexto en cada endpoint sensible.

Resultado La navegación orienta al usuario y el servidor conserva la autoridad.

Área involucrada Autenticación y seguridad.

Aprendizaje La experiencia y la seguridad se complementan, pero no son la misma cosa. Vue puede evitar una pantalla frustrante; PHP debe evitar una operación no autorizada.

29.3 Riesgo de mezcla entre gimnasios

Problema Un usuario asociado a varios gimnasios podía operar recursos fuera del contexto esperado si el servidor aceptaba IDs del cliente.

Causa Alcance multi-tenant incompleto.

Solución aplicada El gimnasio activo se almacena en la sesión, rota sesión/CSRF al cambiar y se vuelve a filtrar en autorización y repositorios.

Resultado Las consultas y mutaciones quedan acotadas al gimnasio y dataset activos.

Área involucrada Backend, sesiones y MySQL.

Aprendizaje En un producto multi-gimnasio, el contexto no puede ser un selector decorativo. Debe acompañar la sesión y aparecer de nuevo en cada consulta sensible.

29.4 Catálogo geográfico difícil de usar

Problema Un panel con filtros y tarjetas podía cubrir demasiado mapa y ocultar gimnasios, especialmente al crecer el contenido.

Causa El mismo espacio intentaba mostrar mapa, filtros y listado completo.

Solución aplicada Se incorporó panel contraíble en escritorio, bottom sheet de tres posiciones en móvil, scroll interno y persistencia local de posición/filtros.

Resultado El mapa puede recuperar espacio sin perder el estado del usuario; los favoritos reales se guardan en MySQL.

Área involucrada Frontend, responsive y geolocalización.

Aprendizaje Agregar controles no siempre mejora una pantalla. El reto fue permitir que la persona escondiera complejidad sin perder sus filtros ni el gimnasio que estaba mirando.

29.5 Cupos y reservas concurrentes

Problema Dos solicitudes simultáneas podían intentar ocupar el último cupo.

Causa Un contador leído y actualizado sin transacción no garantiza exclusión.

Solución aplicada Se separó la clase recurrente de la sesión fechada y se agregaron transacciones, bloqueos, restricciones únicas, idempotencia y lista de espera.

Resultado El cupo no supera la capacidad y una cancelación puede promover al siguiente socio.

Área involucrada Agenda, backend y base de datos.

Aprendizaje Probar un botón una vez no demuestra que el cupo sea seguro. Las reglas de concurrencia se diseñan pensando en dos personas actuando al mismo tiempo.

29.6 Confirmación insegura de pagos

Problema La vuelta a una página de éxito no prueba que el proveedor haya aprobado el pago.

Causa La URL puede abrirse manualmente o llegar antes que la confirmación.

Solución aplicada Se exige webhook firmado, consulta directa a Mercado Pago, referencia externa única e idempotencia.

Resultado La membresía sólo se activa tras confirmación verificable.

Área involucrada Pagos y seguridad.

Aprendizaje Una página de “éxito” comunica navegación, no dinero recibido. La confirmación económica debe provenir del proveedor y poder rastrearse.

29.7 Duplicados en promociones y avisos

Problema Reintentos o cambios de versión podían repetir mensajes.

Causa El envío multicanal necesita identidad estable por destinatario, canal y campaña.

Solución aplicada Se agregaron outbox, claves idempotentes, estados, intentos, recuperación de trabajos y consentimiento versionado.

Resultado Cada entrega conserva trazabilidad y los reintentos no se presentan como nuevos mensajes.

Área involucrada Promociones, notificaciones y datos.

Aprendizaje Reintentar es necesario, pero sin identidad estable puede convertirse en spam.

Una entrega debe poder responder quién, qué campaña, qué canal, qué intento y qué resultado.

29.8 Reportes visuales sin datos reales

Problema Una gráfica o descarga puede parecer correcta aunque use valores inventados.

Causa Prototipos visuales desconectados de la persistencia.

Solución aplicada Finanzas y reportes consumen consultas de MySQL; las exportaciones se generan y autorizan en PHP.

Resultado Métricas y archivos corresponden al gimnasio activo.

Área involucrada Finanzas, reportes y frontend.

Aprendizaje Una gráfica bonita puede ocultar un problema de producto si no se conoce la consulta que la alimenta. Se decidió que cada cifra importante pudiera rastrearse hasta MySQL.

29.9 Estado HTTP 419 bajo Apache

Problema Un CSRF vencido podía terminar registrado como error 500.

Causa El código 419 no siempre producía una línea de estado correcta bajo Apache.

Solución aplicada El responder emite explícitamente `HTTP/1.1 419 Page Expired` y existe una prueba HTTP de regresión.

Resultado El cliente distingue sesión de seguridad vencida de un fallo interno.

Área involucrada API y seguridad.

Aprendizaje Los detalles de infraestructura también forman parte de la experiencia. Un código incorrecto puede hacer que el frontend muestre un mensaje completamente equivocado.

29.10 Concurrencia del rate limiter y reembolsos

Problema Actualizaciones paralelas podían perder intentos o repetir una operación económica.

Causa Secuencias de lectura/escritura no atómicas.

Solución aplicada El límite usa upsert atómico y el repositorio de pagos reserva/lockea operaciones con idempotencia.

Resultado Los controles resisten solicitudes concurrentes con mayor consistencia.

Área involucrada Seguridad y pagos.

Aprendizaje Los contadores y los estados económicos merecen operaciones atómicas. Leer, decidir y escribir por separado deja una ventana que rara vez aparece en una prueba manual.

29.11 Capas modales superpuestas

Problema Varios overlays podían competir por teclado, foco y scroll.

Causa Cada diálogo administraba aisladamente comportamientos globales.

Solución aplicada Se centralizó la pila, Escape, foco, scroll y orden visual en un composable compartido.

Resultado Sólo la capa superior responde y el foco vuelve al control de origen.

Área involucrada Componentes UI y accesibilidad.

Aprendizaje Un modal no es sólo una caja centrada. También administra teclado, foco, scroll, orden visual y retorno al punto de partida.

29.12 Dependencia vulnerable de runtime

Problema La auditoría npm detectó una vulnerabilidad alta transitiva en `nanoid`.

Causa Una versión heredada por la cadena de build.

Solución aplicada Se fijó el override `3.3.18` y se añadió `npm audit --omit=dev --audit-level=high` a CI.

Resultado La auditoría de runtime queda automatizada.

Área involucrada Frontend e integración continua.

Aprendizaje Una dependencia transitiva también es parte del producto. La revisión de seguridad debe repetirse en CI porque el árbol puede cambiar aunque el código de la aplicación no lo haga.

29.13 Qué patrón compartían estos problemas

La mayoría apareció al cruzar una frontera: frontend con backend, usuario con gimnasio, intención con confirmación externa o interacción visual con accesibilidad. Dentro de una sola pantalla todo podía parecer correcto; el fallo surgía cuando la acción debía conservar sentido al atravesar varias capas.

El criterio que mejor funcionó fue formular invariantes antes de pensar en componentes: “una persona no puede reservar dos veces”, “un pago no se aprueba por una URL”, “un empleado no ve otro gimnasio”, “un mensaje no se duplica por reintento”. Después se distribuyó cada defensa en la capa adecuada.

29.14 Problemas típicos a prevenir —no registrados como incidentes del proyecto

Los siguientes riesgos son habituales en una aplicación de este tipo, pero **no se presentan como problemas que hayan ocurrido en GymTrack**. Se incluyen para orientar mantenimiento, pruebas y despliegue.

Documentar estos riesgos ahora evita contar en el futuro una historia que pudo prevenirse con una comprobación sencilla.

Riesgo típico	Cómo podría manifestarse	Prevención o comprobación recomendada
Consultas N+1	Un listado rápido con cinco filas se vuelve lento con cientos.	Medir consultas, usar joins o cargas agrupadas y probar volúmenes realistas.
Índices insuficientes	Filtros administrativos y finanzas consumen CPU creciente.	Revisar EXPLAIN , cardinalidad e índices compuestos por gimnasio, estado y fecha.
Sesiones locales al escalar	Un usuario parece cerrar sesión al caer en otra instancia.	Usar store compartido y comprobar afinidad/replicación antes de escalar horizontalmente.
Entrega deficiente de correo	Mensajes aceptados localmente no llegan o caen en spam.	Configurar SPF, DKIM, DMARC, rebotes y métricas del proveedor.
Reintentos tardíos de webhook	El proveedor confirma después de que la persona cerró la pantalla.	Mantener estados pendientes, worker/reconciliación e idempotencia.
Backup no restaurable	Existe un archivo de copia, pero no puede recuperarse a tiempo.	Ejecutar simulacros de restauración y medir RPO/RTO.
Caché obsoleta	La interfaz muestra permisos o datos anteriores.	Invalidar por contexto, versionar claves y evitar cachear respuestas privadas sin control.
Límites de tiles	El mapa recibe bloqueos por uso intensivo.	Respetar política de OpenStreetMap, atribución, caché permitida y evaluar proveedor dedicado.
Zona horaria o cambio estacional	Calendario e ICS desplazan una clase.	Persistir zona IANA del gimnasio y probar conversiones cerca de cambios horarios.
Crecimiento de archivos	Fotos y exportaciones ocupan el volumen disponible.	Aplicar límites, expiración, limpieza segura y monitoreo de almacenamiento.
Deriva de	Un entorno cree tener una	Ejecutar schema-check , respaldar y aplicar

Riesgo típico	Cómo podría manifestarse	Prevención o comprobación recomendada
migraciones	estructura que otro no posee.	el mismo procedimiento de despliegue.
Falta de alertas	Un error existe durante horas sin que nadie lo vea.	Medir salud, errores 5xx, colas, webhooks, espacio y latencia con alertas accionables.

30 · Trabajo del equipo y trazabilidad

30.1 Una experiencia de trabajo no lineal

El proyecto no avanzó como una lista que se completa una sola vez. Varias decisiones parecían suficientes mientras GymTrack era una maqueta y tuvieron que revisarse al conectar datos reales. Un guard de navegación parecía proteger una sección hasta que se consideró una llamada directa a la API. Una clase con un contador parecía resolver reservas hasta que se pensó en dos solicitudes simultáneas. Un panel geográfico parecía completo hasta que los controles terminaron cubriendo aquello que debía mostrar.

Ese proceso de revisión no fue una señal de fracaso, sino la diferencia entre describir un sistema y hacerlo funcionar. El equipo fue incorporando controles a medida que entendía mejor los límites del producto: quién puede decidir, dónde vive un estado, qué ocurre al repetir una acción y cómo se explica una dependencia todavía no configurada.

30.2 Coordinación y liderazgo — Leandro González

Como líder del proyecto, Leandro tiene la responsabilidad de mantener una dirección común entre áreas que trabajan con ritmos y problemas distintos. La coordinación incluye ordenar prioridades, detectar dependencias, decidir qué debe estar estable para la presentación y evitar que una mejora aislada rompa el recorrido completo.

En un proyecto de estas características, liderar no consiste únicamente en repartir tareas. También implica reunir decisiones: acordar con backend y base de datos qué estados son válidos, confirmar con frontend cómo se comunican, y asegurar con documentación que el resultado pueda explicarse sin exagerar lo que funciona.

30.3 Backend — Máximo Díaz

Máximo es responsable del backend PHP. Su área recibe acciones que llegan desde el navegador y debe convertirlas en operaciones válidas, autorizadas y trazables. Allí se resuelven autenticación, sesiones, CSRF, permisos, contexto de gimnasio, validaciones, pagos, notificaciones, archivos y contratos de API.

El backend ocupa una posición incómoda pero fundamental: no puede confiar completamente en la interfaz y tampoco debería trasladar toda la lógica a SQL. Su trabajo es coordinar capas, devolver errores comprensibles y asegurarse de que una operación externa —como un pago— sólo se confirme cuando exista evidencia suficiente.

30.4 Frontend — Emilio Escobar

Emilio está encargado del frontend Vue. Su responsabilidad es transformar reglas y datos en una experiencia que ayude a actuar y entender el estado del sistema. Esto abarca navegación, componentes, stores, formularios, responsive, accesibilidad, mapa y comunicación con la API.

La dificultad no estuvo sólo en “hacer pantallas”, sino en representar honestamente cargas, vacíos, errores, permisos y funciones condicionadas. El explorador geográfico fue un ejemplo claro: el objetivo no era mostrar más controles, sino dejar visibles mapa y gimnasios y permitir que la persona guardara su forma de explorarlos.

30.5 Base de datos — Santiago Cáceres

Santiago es responsable de la base de datos MySQL. Su área convierte reglas del negocio en estructuras que conserven integridad incluso cuando varias operaciones ocurren al mismo tiempo. Las migraciones, relaciones, restricciones, índices, estados e historiales forman parte de esa responsabilidad.

El crecimiento desde un modelo inicial de diez tablas a un esquema de 49 tablas exigió distinguir conceptos y preservar compatibilidad. La base no es un depósito final: participa activamente en evitar duplicados, aislar gimnasios, conservar eventos y permitir reportes verificables.

30.6 Documentación — Magdalena Belmonti e Hiliana Pereira

Magdalena e Hiliana son las documentadoras del proyecto. Elaboraron el documento de base con la introducción, el análisis del problema, los objetivos, los requisitos RF-01 a RF-20, la planificación, las rutas previstas, la identidad visual y el modelo relacional inicial. Ese trabajo dio al equipo un lenguaje común antes de que la implementación alcanzara su forma actual.

En esta versión, su contenido se fusiona con la evidencia del repositorio. Documentar no significa copiar nombres de archivos: implica explicar por qué existe cada módulo, separar lo estable de lo condicionado y transformar detalles técnicos en recorridos que pueda seguir una persona que no participó del desarrollo.

30.7 Entregas entre áreas

Entrega	Área que la inicia	Área que la completa	Qué se valida en conjunto
Contrato de un flujo	Liderazgo/ producto	Frontend, backend y datos	Actor, estados, permisos y resultado observable.
Endpoint	Backend	Frontend	Método, cuerpo, códigos, errores e idempotencia.
Migración	Base de datos	Backend	Compatibilidad, índices, relaciones y rollback.
Pantalla	Frontend	Documentación y liderazgo	Jerarquía, estados, lenguaje y recorrido.
Integración externa	Backend	Frontend y operación	Credenciales, confirmación real, fallos y soporte.
Documento final	Documentación	Todo el equipo	Exactitud, alcance y explicación comprensible.

30.8 Trazabilidad responsable

Los cargos permiten explicar responsabilidades, pero no prueban autoría exclusiva de cada archivo o línea. Por eso este documento no inventa citas personales ni atribuye una corrección concreta a una persona sin registro verificable. La contribución se describe por área, y el resultado se reconoce como trabajo colectivo.

31 · Herramientas de trabajo utilizadas

- **Visual Studio Code:** edición del frontend, backend, SQL, documentación y configuración.
- **Git y GitHub:** control de versiones, respaldo remoto e integración continua.
- **Docker y Docker Compose:** entornos reproducibles y separación de servicios.
- **Herramientas del navegador:** inspección de red, consola, responsive, accesibilidad y almacenamiento.
- **MySQL y PDO:** persistencia, consultas preparadas, transacciones y comprobación del modelo.
- **Vitest y Vue Test Utils:** pruebas de unidades y componentes frontend.
- **Playwright y Axe:** recorridos de navegador y controles automatizados de accesibilidad.
- **Scripts PHP y Shell:** contratos, seguridad, concurrencia, datasets y preflight productivo.
- **Leaflet y OpenStreetMap:** pruebas cartográficas, marcadores, clusters y geolocalización.

La herramienta principal de verificación es el propio contrato del sistema: una pantalla, un endpoint, una escritura o consulta y una prueba que permitan comprobar el mismo resultado desde perspectivas distintas.

32 · Evolución del proyecto

1. **Base académica.** Se definieron problema, objetivos, stack, arquitectura de tres capas y módulos básicos.
2. **Experiencia pública.** La portada, catálogo y responsive dejaron de ser una maqueta y pasaron a componentes reutilizables.
3. **Identidad segura.** Se incorporaron correo verificado, recuperación, sesiones persistentes, roles y contexto.
4. **Operación multi-gimnasio.** Se conectaron gimnasios, sedes, personas, planes, membresías y permisos por asociación.
5. **Agenda real.** Las clases recurrentes se separaron de sesiones fechadas con cupos, espera y asistencia.
6. **Economía y reportes.** Pagos, Mercado Pago, finanzas, reembolsos y exportaciones pasaron a datos reales.

7. **Exploración geográfica.** El mapa sumó clusters, geolocalización privada, panel adaptable y preferencias guardadas.
8. **Comunicación.** Se añadieron calendario manual, promociones, consentimiento, avisos y worker.
9. **Experiencia completa del socio.** Perfil, favoritos, progreso, medidas y carné completaron el flujo personal.
10. **Preparación productiva.** Se incorporaron CI, preflight, imágenes productivas, hardening y controles de concurrencia.

33 · Limitaciones actuales

- WhatsApp no tiene proveedor real y permanece desactivado.
- Google Calendar no dispone de OAuth ni sincronización bidireccional.
- Analítica avanzada y recomendaciones personalizadas están desactivadas.
- Mercado Pago, SMTP y Turnstile requieren credenciales y validación en staging.
- Los textos legales son provisionales y necesitan revisión jurídica.
- El verificador del carné no usa cámara; acepta token pegado o lector USB tipo teclado.
- El PDF de reportes resume hasta 45 filas; Excel contiene el detalle.
- TLS termina fuera de Docker Compose, delante de Nginx.
- Las sesiones PHP usan volumen local; para escalar horizontalmente se necesita un store compartido.
- Backups, restauración y monitoreo externo son responsabilidades operativas pendientes de ensayo.
- Existe cobertura automatizada, pero no una certificación formal de accesibilidad o seguridad.

34 · Mejoras futuras

Prioridad recomendada después de la presentación:

1. revisión legal y de privacidad;
2. staging con Mercado Pago, SMTP, Turnstile y webhook real;
3. simulacro documentado de backup y restauración;
4. monitoreo, alertas y rotación periódica de secretos e imágenes;
5. store compartido de sesiones si se despliega más de una instancia;
6. pruebas manuales con lector de pantalla y dispositivos reales;
7. OAuth de Google Calendar con actualización y cancelación;

8. proveedor de WhatsApp con consentimiento y plantillas aprobadas;
9. analítica avanzada y recomendaciones sólo después de estabilizar datos y operación.

35 · Instalación, demostración y producción

35.1 Desarrollo

```
cp .env.example .env
docker compose up -d --build
docker compose ps
```

Servicios por defecto: Vue en <http://localhost:5173>, PHP en <http://localhost:8080> y MySQL en localhost:3306.

35.2 Demostración

```
APP_ENV=demo
SEED_DEMO_DATA=true
DEMO_DATASET_NAME=gytrack-presentation
DEMO_USER_PASSWORD=una-clave-local-segura
ALLOW_DEMO_DATA_IN_PRODUCTION=false

docker compose up -d --force-recreate backend
docker compose exec -T backend php console.php seed:demo
```

35.3 Producción

El despliegue productivo exige [.env.production](#) fuera de Git, URLs HTTPS, secretos fuertes, correo, Turnstile, MySQL y Mercado Pago. El preflight falla si detecta demo, cookies inseguras, banderas beta o credenciales incompletas.

```
scripts/production-preflight.sh .env.production
docker compose --env-file .env.production -f docker-compose.prod.yml build
docker compose --env-file .env.production -f docker-compose.prod.yml up -d
```

Las bases existentes deben migrarse antes del arranque; [schema-check](#) verifica todas las versiones requeridas.

36 · Conclusiones

GymTrack evolucionó desde una propuesta académica de gestión de gimnasio hasta una aplicación web con flujos conectados, separación por rol y gimnasio, persistencia relacional, seguridad transversal y una experiencia coherente entre visitante, socio y administración.

La versión 1.0 es adecuada para una presentación funcional porque los módulos principales no dependen de datos inventados en Vue y pueden demostrarse mediante un dataset MySQL aislado. El producto también comunica sus límites: no presenta como terminadas las integraciones que necesitan proveedor, OAuth o revisión externa.

El valor técnico principal no está sólo en la cantidad de pantallas, sino en las decisiones de integridad: autorización en servidor, idempotencia, transacciones, consentimiento, trazabilidad, estados honestos y aislamiento de datos. Esas decisiones permiten continuar el proyecto sin reescribir el stack original.

37 · Fuentes verificadas

Esta documentación se contrastó con:

- [README.md](#) y [DESIGN.md](#);
- documento de base **Documentación.pdf**, elaborado por Magdalena Belmonti e Hiliana Pereira, leído y revisado visualmente en sus 21 páginas;
- diagrama actual [GymTrack_Diagrama.pdf](#) y su fuente SVG;
- rutas Vue y PHP;
- vistas, componentes, stores y servicios frontend;
- controladores, middleware, repositorios y servicios backend;
- esquema y migraciones MySQL;
- seeders y variables de entorno;
- Docker Compose de desarrollo y producción;
- pruebas Vitest, Playwright, PHP y Shell;
- flujo de integración continua.

37.1 Límites de la evidencia

Las responsabilidades generales del equipo fueron confirmadas para esta actualización: Leandro González lidera el proyecto; Máximo Díaz responde por backend; Emilio Escobar, por frontend; Santiago Cáceres, por base de datos; y Magdalena Belmonti e Hiliana Pereira, por documentación. El historial disponible no permite atribuir con fiabilidad cada archivo o línea individual, por lo que no se hacen afirmaciones de autoría exclusiva.

Tampoco se afirma que las integraciones condicionadas hayan sido verificadas con credenciales reales de producción. Los problemas de la sección 29.1 a 29.12 se presentan como dificultades y decisiones registradas en el proyecto; los riesgos de 29.14 están marcados expresamente como preventivos y no como incidentes ocurridos.

Todo lo demás se presenta según el código, las migraciones, la configuración y las pruebas del repositorio al 27 de agosto de 2026.

GymTrack · Proyecto final de Tecnología Web Aplicada · Equipo: Leandro, Máximo, Emilio, Santiago, Magdalena e Hiliana · 2026